

Client-Side Scripting II



Presented by: Carlo Mamo

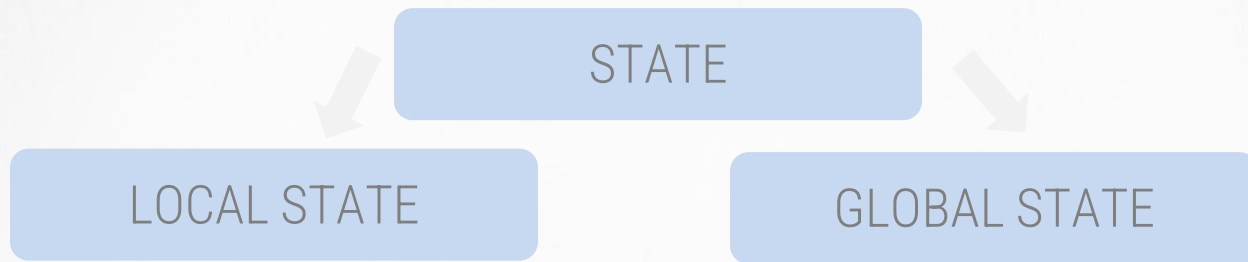
Bachelors of Science (Hons) (MQF Level 6)

Chapter 2 – State Management

Contents

- ▶ Local State versus Global State
- ▶ What is Pinia?
- ▶ Creating and using a store
- ▶ Example
- ▶ Exercise

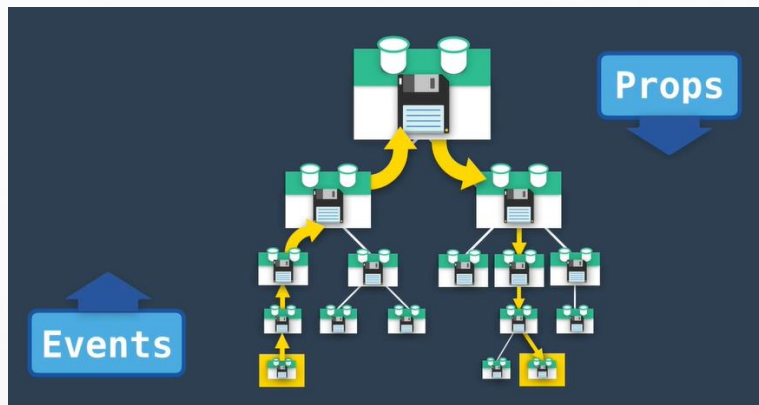
Local state versus global state



- ▶ **Local state** is the data you manage **inside one component which only affects this ONE component** and possibly props of a child component e.g. user input into input elements or a button that shows or hides a container.
- ▶ **Global data** is state that affects **multiple components** possibly the entire app so data that lives in different places e.g. authentication status or shopping cart items presented in different parts of the app.

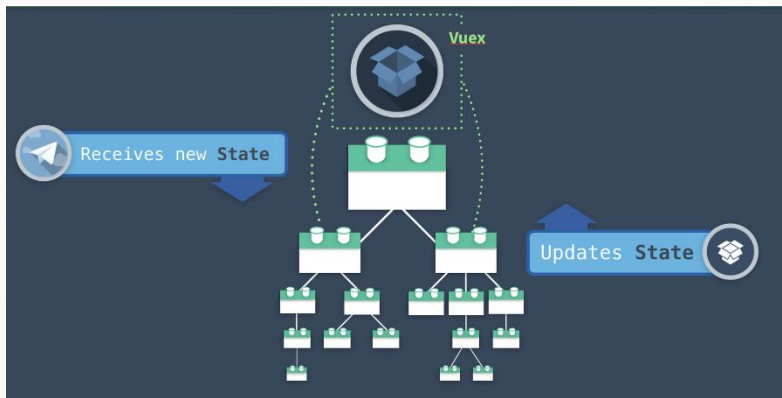
Default Communication

- ▶ Without a state management solution such as VUEX or Pinia each component can have its own version of state but if one component changes, and another component is also using that state we need to communicate that change.
- ▶ The default way is to communicate **EVENTS UP** and passing **PROPS DOWN** to share data **BUT this can become overly complicated.**



Consolidating data in one place

- ▶ Data can be **consolidated in one place** instead of being in each component by having one location that contains the current state of our entire application.
- ▶ This is what VUEX or Pinia provides. All components will have access to this global state which is **reactive**. Hence when one component updates the state other components that are using that same state automatically receive that new data.



Full state management pattern and library

- ▶ However, **consolidating data in one place is not enough**. When different components change data from different locations state changes might be unpredictable and untraceable.
- ▶ This is why Vuex and Pinia provide a **full state management pattern** for a simple and standardized way to make state changes.

What is Pinia?

- ▶ As bigger and bigger applications are built managing the application state (data) can become more challenging.

State = **Reactive data** (data that may change and such changes trigger something on the screen) and VUE.js is all about this reactivity.

- ▶ **Pinia** is about having a better state management solution which can be used to share data properly across multiple components.

What is Pinia?

- ▶ **Pinia** is a state management solution for Vue.js
- ▶ **Pinia** allows us to store global state in “stores” and that state can be read from everywhere on our application
- ▶ When that state value changes inside that store any component that uses that state will automatically get that updated value.
- ▶ It is recommended that Pinia is not added with **small projects of around 5 to 10 components**. If project is projected to grow it might make sense to start of with Pinia.

Example

App.vue (**parent**)

ModalsView.vue (**direct child of App.vue**)

Modal.vue (**direct child of ModalsView.vue**)

```
const userData = reactive({  
  name: "Joe",  
  username: "joeborg"  
})
```

- Assuming userData object is in App.vue to pass it to its descendants we had to use props in both ModalsView.vue and then in Modal.vue.

Example

► Create a new VUE application

```
C:\Users\Carlo\Desktop\VUE\CSS_2\Examples_Exercises>npm init vue@latest
```

Vue.js - The Progressive JavaScript Framework

```
✓ Project name: ... storePiniaEx1
✓ Package name: ... storepiniaex1
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add an End-to-End Testing Solution? » No
✓ Add ESLint for code quality? ... No / Yes
```

```
> node_modules
> public
> src
  > assets
  > components
  > stores
    counter.js
  App.vue
  main.js
```

```
main.js > ...
import './assets/main.css'

import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

const app = createApp(App)

app.use(createPinia())

app.mount('#app')
```

Example

- Under the stores folder a counter.js file shown below on the left will be automatically generated. Change that with the code on the right. In addition make sure to remove all files under the components folder.

stores > JS counter.js > ...

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const doubleCount = computed(() => count.value * 2)
  function increment() {
    count.value++
  }

  return { count, doubleCount, increment }
})
```

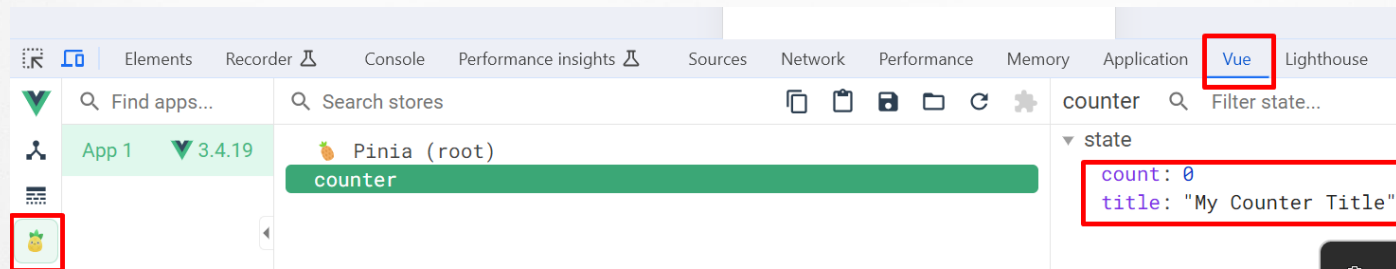
```
export const useCounterStore = defineStore({
  id: "counter",
  state: () => ({
    count: 0,
    title: "My Counter Title",
  }),
});
```

Example

- ▶ To use useCounterStore just import in App.vue and assign it to a variable:

```
<script setup>  
import { useCounterStore } from "@/stores/counter";  
const counter = useCounterStore();  
</script>
```

- ▶ In Vue DevTools we can see the state properties:



Example

- ▶ To access the count and title properties we can do the following:

```
<h3>{{ counter.title }}</h3>  
<span class="counter">{{ counter.count }}</span>
```

Example – Adding Actions to the store

- ▶ Actions **are methods which can access the properties in our state and modify them**. We will add 2 actions in the counter.js file to increase and decrease the counter. To access the actions you simply do: **counter.increaseCounter()** and pass a parameter.

```
actions: {  
  increaseCounter() {  
    this.count++  
  },  
  decreaseCounter() {  
    this.count--  
  }  
}
```

Exercise

- ▶ Add the following functionality in App.vue. User should be able to decrease the counter by 2 or 1 and also increase the counter value by 1 or 2.

My Counter Title

4

Decrease by 2

Decrease by 1

4

Increase by 1

Increase by 2

Adding getters to the Store

- ▶ Getters allow us to get a value from the state and then modify it in some way or generate something based on that value and then return it making it available to any component which is using this store.
- ▶ Getters are the **equivalent of computed properties** but in state management.
- ▶ So the returned values of getters is cached and only updated whenever their dependencies change.

Exercise

- ▶ To add getters you need to add another object getters in counter.js:
 - Then set a property oddOrEven and pass state as a parameter in this method

```
getters: {  
  oddOrEven: (state) => {  
    if (state.count % 2 === 0) return "even";  
    return "odd";  
  },  
},
```

<p>This counter is {{ counter.oddOrEven }}</p>

Data, actions and getters

- ▶ The state is the central part of your stores and used to define data that is accessible through all components of the application.

```
// Example state with an items array return  
state: () => {  
  return {  
    items: [],  
  }  
}
```

```
import { defineStore } from 'pinia'  
import { usePostStore } from './post'  
  
export const useCommentStore = defineStore({  
  id: 'comment',  
  state: () => ({  
    comments: []  
  }),  
})
```

Data, actions and getters

- ▶ Actions in your store are synonymous with methods for your component except they mutate the store data instead of a component's local data. These methods can be asynchronous and hence can be used to fetch data. 2 Examples below.

```
// Example addItem Action that adds a new the items  
to the array  
actions:{  
  addItem(item){  
    this.items.push(item);  
  }  
}
```

```
actions: {  
  async fetchComments() {  
    this.comments = await fetch('https://jsonplaceholder.typicode.com/comments')  
      .then((response) => response.json())  
  }  
}
```

Data, actions and getters

- ▶ Getters are synonymous with computed properties in a component. Its purpose is to return a computed value based on the store's state. By default a getter receives the state as a first argument. 2 Examples below.

```
// Example count Getter that return the length of the  
items array  
getters:{  
  count(){  
    return this.items.length;  
  }  
}
```

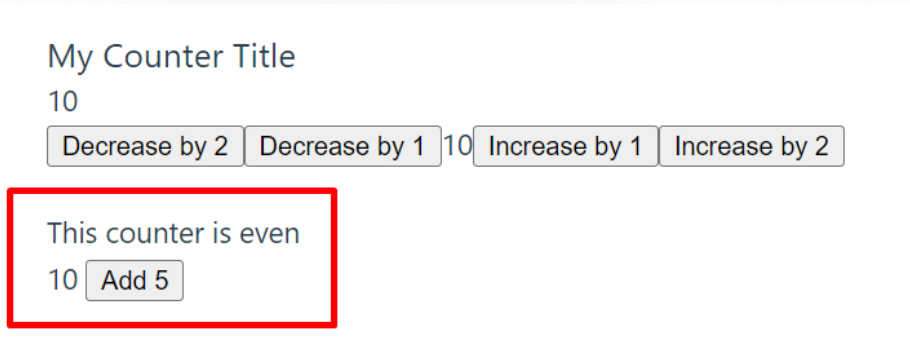
```
getters: {  
  getPostComments: (state) => {  
    const postSore = usePostStore()  
    return state.comments.filter((post) => post.postId === postSore.post.id)  
  }  
},
```

Store can be used anywhere in our App

- ▶ Once you import the store e.g. `import {useCounterStore} from '@stores/counter'` and declare a constant e.g. `const counter = useCounterStore()` you can use the counter anywhere in the App.

- ▶ **Exercise**

Create a new component under the components folder. Name the component **DisplayCounter.vue**. In this component create a button which uses the `increaseCounter` action to add 5 to the counter value. Include this component in `App.vue`. Note that counter in store is always up-to-date regardless where we are in our App since its global is being modified.



References

Vuejs.org. 2021. *Vue.js*. [online] Available at: <<https://vuejs.org/>> [Accessed 7 February 2021].

Schweiger, P., 2021. *Vue Mastery*. [online] Vue Mastery. Available at: <<https://www.vuemastery.com/>> [Accessed 7 February 2021].

Vue School. 2021. *Learn Vue.js from core-team members and industry experts at Vue School*. [online] Available at: <<https://vueschool.io/>> [Accessed 7 February 2021].

Udemy. 2021. *Develop with VueJS 2 (Complete Vue.js Router and Vuex Course)*. [online] Available at: <<https://www.udemy.com/course/vuejs-2-the-complete-guide/>> [Accessed 7 February 2021].

New Course! - Vue JS 3: Composition API (with Pinia & Vite). (n.d.). [Www.youtube.com](https://www.youtube.com). Retrieved March 6, 2024, from <https://www.youtube.com/watch?v=iOUcYu5P158>



Any questions?

THANK YOU

Email: carlo.mamo@mcast.edu.mt

